Fundamentos de GitHup

Presentado por; Anny Sofia Beltrán García

Presentado a; Carlos Andrés Gómez Vasco

Medicina veterinaria

Universidad agraria de Colombia

UNIAGRARIA

Ciencia de datos

2026

CAPÍTULO 2. FUNDAMENTOS DE GIT

Introducción

Git es un sistema de control de versiones distribuido que permite registrar, organizar y administrar los cambios realizados en los archivos de un proyecto. Su utilización facilita el seguimiento de las modificaciones, la recuperación de versiones anteriores y el trabajo colaborativo entre diferentes personas.

En este capítulo se explican los fundamentos del funcionamiento de Git, la creación de repositorios, el registro de cambios, la consulta del historial, la recuperación de archivos y la conexión con repositorios remotos como GitHub

2.1. Obtener un repositorio Git

Un repositorio es el espacio donde Git almacena los archivos de un proyecto y el historial de sus modificaciones

Existen dos formas principales de obtener un repositorio:

1. **Crear un repositorio nuevo:** se utiliza cuando se comienza un proyecto desde cero. Para hacerlo, se ingresa a la carpeta del proyecto desde la terminal y se ejecuta: git init Este comando inicializa Git en la carpeta seleccionada y crea la estructura interna necesaria para comenzar a registrar los cambios
2. **Clonar un repositorio existente:** permite obtener una copia de un proyecto que ya se encuentra alojado en una plataforma remota, como GitHub

   El comando utilizado es: git clone URL_DEL_REPOSITORIO

Se debe reemplazar URL_DEL_REPOSITORIO por la dirección real del repositorio

- Por ejemplo: git clone https://github.com/usuario/proyecto.git

Al ejecutar este comando, Git descarga los archivos y el historial del repositorio remoto al computador

2.2. Grabar cambios en un repositorio

Cuando se modifican archivos dentro de un proyecto, Git permite registrar esos cambios mediante un proceso organizado en tres estados principales:

- **Directorio de trabajo:** contiene los archivos que el usuario puede modificar.
- **Área de preparación:** contiene los cambios seleccionados para incluir en la siguiente confirmación
- **Repositorio:** almacena las confirmaciones o commits realizados

1. Para revisar el estado de los archivos se utiliza: **git status**
2. Para agregar un archivo específico al área de preparación:

git add nombre_del_archivo

3. Para agregar todos los cambios del directorio actual:

git add

Después de preparar los archivos, se registra una confirmación mediante:

git commit -m

El mensaje debe explicar de manera breve qué modificaciones se guardaron.

Ejemplo con este capitulo:

Si se creó un documento llamado Capitulo_2_Fundamentos.docx, se puede utilizar:

git add Capitulo_2_Fundamentos.docx

Luego:

git commit -m "Agrega el capítulo 2 sobre fundamentos de Git"

De esta manera, los cambios quedan registrados en el historial del repositorio local

2.3. Ver el historial de confirmaciones

Git permite consultar las confirmaciones realizadas en un proyecto. Esta función es importante porque permite conocer qué cambios se han registrado y en qué momento.

- El comando principal es: **git log**

Este comando muestra información de las confirmaciones, como el identificador, el autor, la fecha y el mensaje

- Para consultar el historial de manera resumida se utiliza: **git log --oneline**

Esta opción facilita la lectura del historial, por ello consultar el historial permite verificar que los cambios se hayan registrado correctamente

2.4. Deshacer cosas

Durante el desarrollo de un proyecto pueden presentarse errores o modificaciones que se desean corregir. Git ofrece diferentes herramientas para recuperar o modificar el estado de los archivos, algunos comandos importantes son:

- Consultar los cambios realizados: **git diff**

Permite visualizar las diferencias entre los archivos modificados y la última versión registrada

- Retirar un archivo del área de preparación: **git restore --staged**

  nombre_del_archivo

Este comando retira el archivo del área de preparación, pero conserva sus modificaciones en el directorio de trabajo.

- Descartar modificaciones locales de un archivo: **git restore nombre_del_archivo**

Este comando recupera la versión registrada del archivo y descarta los cambios locales no confirmados. Se debe utilizar con cuidado, porque los cambios descartados pueden perderse también es importante revisar el estado del repositorio antes de utilizar comandos que puedan eliminar modificaciones

2.5. Trabajar con repositorios remotos

Un repositorio remoto es una versión del proyecto alojada en un servidor o plataforma en línea, como GitHub que permite compartir los archivos y sincronizar el trabajo entre diferentes computadores y colaboradores

- Para consultar los repositorios remotos configurados se utiliza: **git remote -v**
- Para conectar un repositorio local con uno remoto se utiliza: **git remote add origin URL_DEL_REPOSITORIO**
- Para enviar los commits locales al repositorio remoto se utiliza: **git push -u origin main**
- Para descargar e integrar cambios del repositorio remoto se utiliza: **git pull**

Estos comandos permiten mantener sincronizadas las versiones locales y remotas del proyecto

2.6. Etiquetado

Las etiquetas o *tags* permiten identificar puntos específicos del historial de un proyecto. Son útiles para señalar versiones importantes, como una primera entrega o una versión final

- Para crear una etiqueta sencilla se utiliza: **git tag v1.0**

- Para consultar las etiquetas existentes: **git tag**

Las etiquetas ayudan a organizar las versiones del proyecto y facilitan la identificación de entregas importantes

2.7. Alias de Git

Los alias permiten crear nombres cortos para comandos que se utilizan frecuentemente. Su objetivo es facilitar y agilizar el trabajo en la terminal. Por ejemplo, se puede crear un alias para consultar el historial resumido**: git config --global alias.hist "log --oneline".** Después de configurarlo, se puede ejecutar: **git hist.** En lugar de escribir el comando completo **git log --oneline**

Los alias son opcionales y pueden personalizarse según las necesidades del usuario.

2.8. Ejercicio práctico: registrar y enviar cambios

Esto permite aplicar los conocimientos del capítulo utilizando un repositorio de Git

Procedimiento:

1. Abrir CMD o Git Bash
2. Ingresar a la carpeta local del proyecto
3. Ejecutar git status para revisar el estado del repositorio
4. Crear o modificar un archivo dentro del proyecto
5. Ejecutar git add . para preparar los cambios
6. Ejecutar git status para comprobar que los archivos están preparados
7. Registrar los cambios con un commit**: git commit -m "Registra cambios del proyecto"**
8. Consultar el historial: **git log --oneline**

9. Enviar los cambios a GitHub: **git push**

Con esto queremos comprender cómo Git registra los cambios, permite consultar el historial y sincroniza el trabajo local con un repositorio remoto

Conclusiones

Los fundamentos de Git permiten comprender cómo se administran los archivos y las modificaciones de un proyecto mediante un sistema de control de versiones. La creación y clonación de repositorios, el uso de los comandos **git add** y **git commit**, y la consulta del historial son procesos esenciales para mantener un registro organizado del trabajo.

Además, la conexión con repositorios remotos mediante comandos como **git push** y **git pull** facilita la colaboración y el intercambio de información entre los integrantes de un equipo

Referencias bibliográficas

1. Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/es/v2
2. Git. (s. f.). *Git*. https://git-scm.com/